

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Работа с файлами	2
Чтение файлов	2
Чтение бинарных (двоичных) файлов	5
Запись в файл	5
Таблица аргументов работы с файлами:	6

Работа с файлами

Чтение файлов

В практике очень часто приходится работать с данными, поступающими из файлов. Это могут быть файлы с расширением txt, cvs, html и др.

Создадим текстовый файл test.txt с этим текстом:

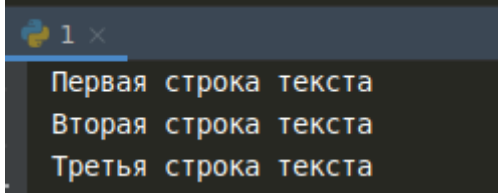
Первая строка текста

Вторая строка текста

Третья строка текста

Для чтения сделаем программу и запустим.

```
file = open('test.txt', 'r')
content = file.read()
print(content)
file.close()
```

The screenshot shows a Python script being executed in a Jupyter Notebook. The code defines a file 'test.txt' in read mode, reads its content, prints it, and closes the file. The output of the script is displayed in a separate cell, showing the three lines of text from the file: 'Первая строка текста', 'Вторая строка текста', and 'Третья строка текста'.

При работе с файлами соблюдается некоторая последовательность операций:

- * Открытие файла с помощью метода `open()`
- * Чтение файла с помощью метода `read()` или запись в файл методом `write()`
- * Закрытие файла методом `close()`

После вызова функции `read()` файл будет прочитан как строка в переменную `content`. Путь к файлу может быть как относительным, так и абсолютным. Второй передаваемый аргумент – `mode` устанавливает режим открытия файла в зависимости от того, что мы собираемся с ним делать. Таблица всех режимов находится на последней странице методички.

После того, как мы закончили работу с файлом необходимо закрыть дескриптор файла. Следует **всегда закрывать дескриптор файла**, так как неизвестно, когда и какая именно программа захочет получить к нему доступ. Закрытие файла также поможет сохранить память и избежать появления багов в программе.

Лучше всего пользоваться контекстным менеджером `with`, который не требует освобождения ресурсов, то есть применения `close()`. Закрытие файла происходит автоматически. Перепишем нашу программу с менеджером контекста, и запустим:

```
with open('test.txt', 'r') as file:
    content = file.read()
print(content)
```

```
1 x
Первая строка текста
Вторая строка текста
Третья строка текста
```

Файл получается немного короче, и синтаксис схож с условными операторами – требуется двоеточие с отступом после него. В момент вызова функции open, Python сначала ищет файл в рабочем каталоге (там, где расположен скрипт). Если же файл находится в другом каталоге, то необходимо указать путь, пример в windows “C:\\example\\test.txt”, в линукс '/home/scripts/test.txt'.

Пример с обработкой исключения:

```
try:
    with open('tests.txt', 'r') as file:
        content = file.read()
    print(content)
except:
    print('Ошибка открытия файла')
```

```
1 x
Ошибка открытия файла
```

Как видите, блок try содержит довольно много строк кода, поэтому разумнее будет в таких случаях отказаться от менеджера контекста with и использовать обычную функцию open:

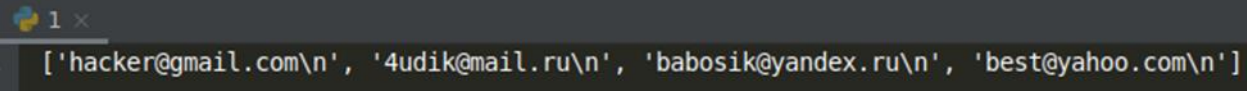
```
In [1]: target_file = 'test_1.txt'
...: try:
...:     file = open(target_file, 'r')
...: except FileNotFoundError:
...:     print('Файл', target_file, 'не найден')
...: else:
...:     content = file.read()
...:     file.close()
...:     print(content)
...:
Файл test_1.txt не найден
```

Нередко может в программе потребоваться список из данных в файле, например список, логинов/паролей, адресов хостов, e-мейлов и т.п. Перепишем наш текстовый файл:

hacker@gmail.com
4udik@mail.ru
babosik@yandex.ru
best@yahoo.com

Метод `readlines()` возвращает содержимое файла в виде **списка строк**.

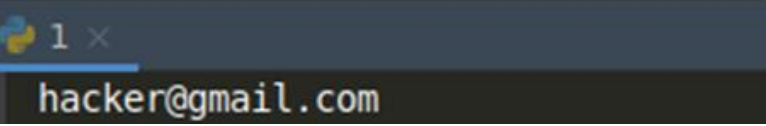
```
with open('test.txt', 'r') as file:
    content = file.readlines()
print(content)
```



```
1 x
['hacker@gmail.com\n', '4udik@mail.ru\n', 'babosik@yandex.ru\n', 'best@yahoo.com\n']
```

В отличие от метода `readlines()`, метод `readline()` читает только одну строку за один вызов.

```
with open('test.txt', 'r') as file:
    content = file.readline()
print(content)
```

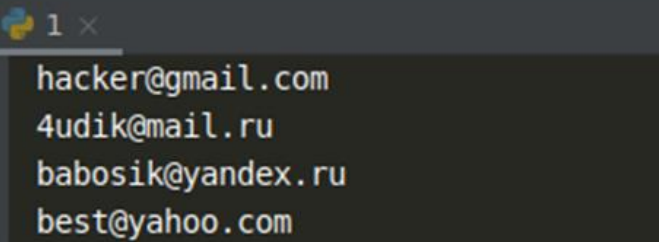


```
1 x
hacker@gmail.com
```

Однако файл может быть слишком большим для того, чтобы поместиться памяти, выделенной программе операционной системой. В этом случае оптимально считывать и обрабатывать файл частями.

Частями прочитать файл можно при помощи цикла, заодно уберём знак переноса строки с помощью метода `strip()`:

```
with open('test.txt', 'r') as file:
    for content in file:
        print(content.strip())
```



```
1 x
hacker@gmail.com
4udik@mail.ru
babosik@yandex.ru
best@yahoo.com
```

Код выше не сохраняет содержимое файла в переменную, мы просто пробегаемся по этому файлу от начала и до конца, печатая строки по одной.

Чтение бинарных (двоичных) файлов

Для чтения бинарных файлов нужно изменить способ доступа к файлу на "rb". В результате мы получаем строку, состоящую из байтов.

```
try:
    with open('/root/koshak.jpg', 'rb') as file:
        content = file.read()
    print(content)
except:
    print('Ошибка открытия файла')
```

1 x

```
b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x01\x00
```

Запись в файл

Режим дозаписи в файл в самом простейшем виде будет выглядеть так:

```
with open('test.txt', 'a') as file:
    file.write('monster@inbox.com' + '\n')
```

```
hacker@gmail.com
4udik@mail.ru
babosik@yandex.ru
best@yahoo.com
monster@inbox.com
```

Каждый вызов метода `write()` запишет строку в конец файла.

Если же аргумент `mode` будет 'w', то запись начнётся с начала файла и он перезапишется полностью:

```
with open('test.txt', 'w') as file:
    file.write('monster@inbox.com' + '\n')
```

```
monster@inbox.com
```

При возникновении проблем с кодировками, вопрос можно решить, указав принудительно нужную кодировку.

```
with open('test.txt', 'w', encoding='utf-8') as file:
    file.write('キングヨ' + '\n')
```

Таблица аргументов работы с файлами:

Режим	Описание
'r'	Открытие файла на чтение (режим по умолчанию)
'w'	Открытие файла на запись, если файл отсутствует, создаётся новый
'x'	Открытие файла на запись, если файл уже существует, получим исключение
'a'	Открытие файла на дозапись, данные добавляются в конец файла
'b'	Открытие файла в двоичном режиме
't'	Открытие файла в текстовом режиме (режим по умолчанию)
'+'	Обновление файла – открытие на чтение и запись